

Implementation and Analysis of BST, AVL and RBT Data Structures

Sebastiano Babich (172249) - Leonardo Mazzon (172587)
- Damiano Netto (171948)

University of Udine, Academic Year 2025 - 2026

Contents

1	Abstract	2
2	Introduction	2
3	Theoretical Background and Implementation	2
3.1	Binary Search Tree (BST)	2
3.2	Adelson-Velsky Landis Tree (AVL)	2
3.3	Red Black Tree (RBT)	3
4	Test Cases Setup	3
5	Results and Discussion	3
6	Conclusion	3

1 Abstract

The goal of the project was to implement and compare some of the data structures studied in the “Algorithms and Data Structures” course at the University of Udine, specifically standard and self-balancing binary search trees. The project focused in particular on the time complexity involved in inserting and deleting data within these structures.

2 Introduction

Search trees are among the most widely used data structures, since they enable high performance in both searching, inserting and deleting data. Specifically, the trees used in our project are binary: each node has at most two children, which can be other nodes or pointers to NULL. The resulting tree is acyclic, and one of its most important characteristics is its height that is defined as the maximum number of nodes on the path from the root to any leaf.

Height is a fundamental property of a tree: keeping its value as small as possible helps ensure efficient operations thanks to more efficient search, insertion and deletion operations. Balanced binary trees are designed to keep this value as low as possible, on the order of the base-2 logarithm of the number of nodes.

3 Theoretical Background and Implementation

3.1 Binary Search Tree (BST)

3.2 Adelson-Velsky Landis Tree (AVL)

AVL trees are a data structure that implements self balancing binary search trees. The search operation in the tree exactly mirrors the search in BST. Insertion initially uses the BST insertion method, with the difference that a self balancing operation is performed afterward.

Specifically, this operation aims to identify imbalances in the tree by comparing the heights of two child nodes. Specifically, the goal is to minimize the total height of the tree, so that search operations remain as close as possible to logarithmic complexity.

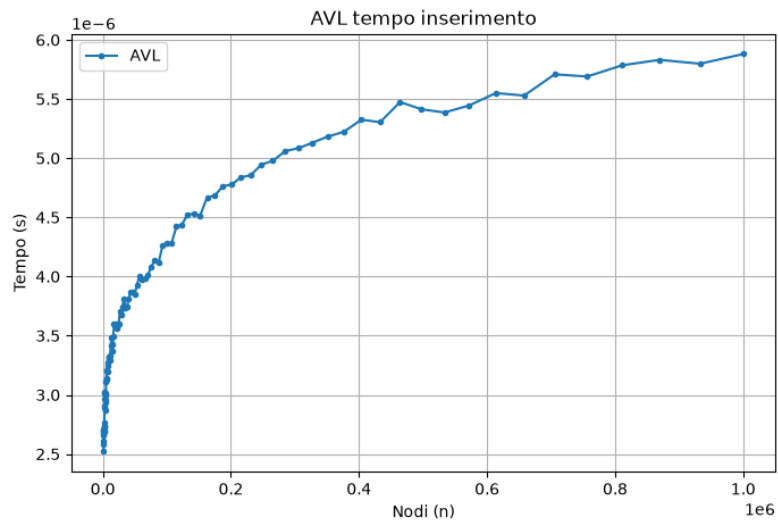


Figure 1: Test eseguito

3.3 Red Black Tree (RBT)

4 Test Cases Setup

For the test cases, we have divided the code into three separate files:

- *setup_test.py*:
- *test_insertion.py*:
- *test_rotations.py*:

5 Results and Discussion

6 Conclusion